

Rezolvare detaliată - Subiect 01

Programare Avansată pe Obiecte în Java
Colocviu - Sesiunea iunie 2024

Rezumat răspunsuri

Exercițiu	Răspuns	Tema principală
I.1	d	overriding, overloading, erori de redefinire
I.2	c	HashSet, equals, hashCode
I.3	b	câmpuri vs. metode la upcasting
I.4	a	propagarea excepțiilor și finally
I.5	c	interfețe funcționale, lambda, metode default

1. Subiectul I - grilă, rezolvare explicată

1.1. Exercițiul I.1 - overriding, overloading și erori

Codul este:

```

1 class A {
2     public A met1() { return new A(); }
3     final void met2() { }
4     public void met3() { }
5     public void met4() { }
6     int met5(int i) { return 5; }
7 }
8
9 class B extends A {
10     public B met1() { return new B(); }
11     public void met2() { }
12     void met3() { }
13     public static void met4() { }
14     int met5() { return 5; }
15 }

```

Analizăm fiecare metodă din clasa B.

Metodă	Explicație	Rezultat
met1()	În A, metoda întoarce A. În B, metoda întoarce B. Deoarece B extinde A, tipul returnat este covariant. Lista parametrilor este aceeași.	suprascriere
met2()	În A, metoda este final. O metodă final nu poate fi suprascrisă în subclasă.	eroare
met3()	În A, metoda este public. În B, metoda are acces implicit/package-private. La overriding nu avem voie să micșorăm nivelul de acces.	eroare
met4()	În A, metoda este non-statică. În B, metoda este statică. O metodă de instanță nu poate fi ascunsă printr-o metodă statică cu aceeași semnătură.	eroare
met5()	În A avem met5(int i), iar în B avem met5(). Lista parametrilor diferă, deci este supraîncărcare, nu suprascriere.	supraîncărcare

Deci avem:

- o suprascriere: met1;
- o supraîncărcare: met5;
- trei erori: met2, met3, met4.

Răspuns corect: **d) o suprascriere, o supraîncărcare, trei erori**

Idee de examen: La overriding: aceeași semnătură, tip returnat identic sau covariant, acces cel puțin la fel de permisiv, metoda din părinte să nu fie final, iar static/non-static trebuie să rămână compatibil.

1.2. Exercițiul I.2 - HashSet, equals și hashCode

Codul este:

```

1 class A {
2     int val;
3     public A(int val) { this.val = val; }
4     public boolean equals(Object obj) { return val != ((A)obj).val; }
5     public int hashCode() { return val / 100; }
6 }
7
8 public class Test {
9     public static void main(String[] args) throws IOException {
10         HashSet<A> hs = new HashSet();
11         hs.add(new A(100));
12         hs.add(new A(10));
13         hs.add(new A(1));
14         hs.add(new A(10));
15         hs.add(new A(100));
16         System.out.println(hs.size());
17     }
18 }

```

Atenție: metoda equals este scrisă intenționat greșit pentru egalitate logică:

```
1 return val != ((A)obj).val;
```

Ea întoarce true când valorile sunt diferite și false când valorile sunt egale. Aceasta încalcă ideea normală de egalitate.

Calculăm hashCode:

Obiect	hashCode = val / 100
new A(100)	1
new A(10)	0
new A(1)	0
new A(10)	0
new A(100)	1

Parcurgem adăugările în HashSet:

1. add(new A(100)): bucket hash 1 este gol, deci se adaugă. Dimensiune: 1.
2. add(new A(10)): bucket hash 0 este gol, deci se adaugă. Dimensiune: 2.
3. add(new A(1)): are hash 0, deci se compară cu A(10). A(1).equals(A(10)) întoarce true, deoarece 1 != 10. Deci setul consideră că există deja un element egal și nu îl adaugă. Dimensiune: 2.
4. add(new A(10)): are hash 0, se compară cu vechiul A(10). A(10).equals(A(10)) întoarce false, deoarece 10 != 10 este fals. Deci este considerat diferit și se adaugă. Dimensiune: 3.
5. add(new A(100)): are hash 1, se compară cu vechiul A(100). A(100).equals(A(100)) întoarce false, deci se adaugă. Dimensiune: 4.

Prin urmare, se afișează:

```
1 4
```

Răspuns corect: c) 4

Idee de examen: În HashSet, două obiecte pot fi considerate egale numai dacă au același hashCode și equals întoarce true. Dacă equals este greșit definit, rezultatul poate părea contraintuitiv.

1.3. Exercițiul 1.3 - câmpuri vs metode la upcasting

Codul este:

```

1 class A {
2     int x = 10;
3     int f(int t) { return x + t; }
4 }
5
6 class B extends A {
7     int x = 30;
8     int f(int t) { return x + 10 * t; }
9 }
10
11 public class Test {
12     public static void main(String[] args) {
13         A ob = new B();
14         System.out.println(ob.f(ob.x));
15     }
16 }

```

Avem:

```
1 A ob = new B();
```

Deci:

- tip declarat al referinței: A;
- tip real al obiectului: B.

Regula importantă:

- câmpurile/atributele se aleg după tipul declarat al referinței;
- metodele de instanță suprascrise se aleg după tipul real al obiectului.

Așadar:

```
1 ob.x
```

se ia din A, deci are valoarea 10.

Apoi:

```
1 ob.f(ob.x)
```

apelează metoda f din B, deoarece metoda este suprascrisă și obiectul real este de tip B.

În metoda din B:

```
1 int f(int t) { return x + 10 * t; }
```

x este câmpul clasei B, deci $x = 30$. Parametrul t primește ob.x, adică 10.
Rezultatul este:

$$30 + 10 \cdot 10 = 130$$

Răspuns corect: **b) 130**

Idee de examen: La `A ob = new B()`, metodele non-statice se aleg după obiectul real, dar câmpurile se aleg după tipul declarat al referinței.

1.4. Exercițiul 1.4 - excepții și finally

Codul este:

```
1 class Test {
2     static String sir = "E";
3
4     void A() {
5         try {
6             sir = sir + "A";
7             B();
8         } catch (Exception e) {
9             sir = sir + "B";
10        }
11    }
12
13    void B() throws Exception {
14        try {
15            sir = sir + "C";
16            C();
17        } catch (Exception e) {
18            throw new Exception();
19        } finally {
20            sir = sir + "D";
21        }
22    }
23
24    void C() throws Exception {
25        throw new Exception();
26    }
27
28    public static void main(String[] args) {
29        Test ob = new Test();
30        ob.A();
31        System.out.println(sir);
32    }
33 }
```

Urmărim pas cu pas:

1. Inițial: `sir = "E"`.
2. Se apelează `A()`.
3. În `A()`, se execută:

```
1 sir = sir + "A";
```

Deci `sir = "EA"`.

4. Se apelează `B()`.
5. În `B()`, se execută:

```
1 sir = sir + "C";
```

Deci `sir = "EAC"`.

6. Se apelează `C()`, care aruncă o excepție.
7. Excepția este prinsă în catch-ul din `B()`, care aruncă o nouă excepție.
8. Înainte ca excepția să plece mai departe spre `A()`, se execută obligatoriu blocul `finally` din `B()`:

```
1 sir = sir + "D";
```

Deci sir = "EACD".

9. Excepția ajunge în A() și este prinsă de catch-ul din A():

```
1 sir = sir + "B";
```

Deci sir = "EACDB".

Se afișează:

```
1 EACDB
```

Răspuns corect: a) EACDB

Idee de examen: Blocul finally se execută chiar dacă în catch se aruncă o altă excepție. Ordinea este: try → catch → finally → propagare mai departe.

1.5. Exercițiul 1.5 - interfață funcțională, lambda, metodă default

Codul este:

```
1 interface Functie {
2     int f(double x);
3     default boolean eval(double x) {
4         return f(x) == x;
5     }
6 }
7
8 public class Test {
9     Functie rf;
10
11     public Test(Functie rf) {
12         this.rf = rf;
13     }
14
15     public boolean eval(double x) {
16         return rf.f(x) == x;
17     }
18 }
```

Analizăm afirmațiile.

- (a) **Adevărat.** Interfața Functie este funcțională. Are o singură metodă abstractă: f(double). Metoda eval este default, deci nu se numără ca metodă abstractă.
- (b) **Fals.** Metoda eval din clasa Test nu rescrie metoda implicită eval din interfața Functie, deoarece Test nu implementează Functie. Are doar un atribut de tip Functie.
- (c) **Adevărat.** Expresia boolean b = new Test(x -> 10).eval(10); este corectă. Lambda x -> 10 implementează int f(double x).
- (d) **Fals.** Expresia boolean b = new Test(x -> x / 100).eval(10); nu este corectă. x este double, deci x / 100 este double, dar metoda f trebuie să întoarcă int.
- (e) **Adevărat.** Clasa anonimă implementează metoda abstractă f(double), deci expresia este corectă.

- (f) **Adevărat.** În expresia cu clasa anonimă, nu se redefinește `eval`, deci se apelează metoda default `eval` din interfața `Funcție`.

Afirmațiile adevărate sunt: (a), (c), (e), (f). În total sunt 4 afirmații adevărate.

Răspuns corect: c 4

Idee de examen: O interfață funcțională are exact o metodă abstractă. Metodele default și static din interfață nu se numără la acest total. O clasă rescrie o metodă default numai dacă implementează interfața respectivă.

2. Subiectul II - clasa TennisATP și prelucrări cu stream-uri

Cerința cere să definim complet clasa TennisATP, apoi să creăm o listă cu cel puțin 3 jucători și să rezolvăm cerințele folosind stream-uri și lambda expresii.

2.1. Clasa TennisATP

```
1 import java.util.Objects;
2
3 public class TennisATP {
4     private String numeJucator;
5     private String nationalitateJucator;
6     private int pozitieATP;
7     private int puncteATP;
8     private int nrTurneeATP;
9
10    public TennisATP() {
11    }
12
13    public TennisATP(String numeJucator, String nationalitateJucator,
14                      int pozitieATP, int puncteATP, int nrTurneeATP) {
15        this.numeJucator = numeJucator;
16        this.nationalitateJucator = nationalitateJucator;
17        this.pozitieATP = pozitieATP;
18        this.puncteATP = puncteATP;
19        this.nrTurneeATP = nrTurneeATP;
20    }
21
22    public String getNumeJucator() {
23        return numeJucator;
24    }
25
26    public void setNumeJucator(String numeJucator) {
27        this.numeJucator = numeJucator;
28    }
29
30    public String getNationalitateJucator() {
31        return nationalitateJucator;
32    }
33
34    public void setNationalitateJucator(String nationalitateJucator) {
35        this.nationalitateJucator = nationalitateJucator;
36    }
37
38    public int getPozitieATP() {
39        return pozitieATP;
40    }
41
42    public void setPozitieATP(int pozitieATP) {
43        this.pozitieATP = pozitieATP;
44    }
45
46    public int getPuncteATP() {
47        return puncteATP;
48    }
49
50    public void setPuncteATP(int puncteATP) {
51        this.puncteATP = puncteATP;
52    }
53 }
```



```

54 public int getNrTurneeATP() {
55     return nrTurneeATP;
56 }
57
58 public void setNrTurneeATP(int nrTurneeATP) {
59     this.nrTurneeATP = nrTurneeATP;
60 }
61
62 @Override
63 public String toString() {
64     return "TennisATP{" +
65         "numeJucator='" + numeJucator + '\'' +
66         ", nationalitateJucator='" + nationalitateJucator + '\'' +
67         ", pozitieATP=" + pozitieATP +
68         ", puncteATP=" + puncteATP +
69         ", nrTurneeATP=" + nrTurneeATP +
70         '}';
71 }
72
73 @Override
74 public boolean equals(Object obj) {
75     if (this == obj) {
76         return true;
77     }
78     if (!(obj instanceof TennisATP)) {
79         return false;
80     }
81     TennisATP other = (TennisATP) obj;
82     return pozitieATP == other.pozitieATP
83         && puncteATP == other.puncteATP
84         && nrTurneeATP == other.nrTurneeATP
85         && Objects.equals(numeJucator, other.numeJucator)
86         && Objects.equals(nationalitateJucator, other.nationalitateJucator)
87 ;
88 }
89
90 @Override
91 public int hashCode() {
92     return Objects.hash(numeJucator, nationalitateJucator,
93         pozitieATP, puncteATP, nrTurneeATP);
94 }

```

2.2. Rezolvarea cerințelor cu stream-uri

```

1 import java.util.ArrayList;
2 import java.util.Comparator;
3 import java.util.List;
4 import java.util.Map;
5 import java.util.TreeMap;
6 import java.util.stream.Collectors;
7
8 public class TestTennisATP {
9     public static void main(String[] args) {
10         List<TennisATP> jucatori = new ArrayList<>();
11
12         jucatori.add(new TennisATP("Novak Djokovic", "Serbia", 1, 9855, 18));
13         jucatori.add(new TennisATP("Carlos Alcaraz", "Spania", 2, 8805, 16));
14         jucatori.add(new TennisATP("Jannik Sinner", "Italia", 3, 8270, 14));
15         jucatori.add(new TennisATP("Andrei Popescu", "Romania", 90, 1200, 8));

```

```

16     jucatori.add(new TennisATP("Mihai Ionescu", "Romania", 75, 2500, 11));
17     jucatori.add(new TennisATP("Ion Radu", "Romania", 60, 3900, 12));
18
19     // a) Jucatorii care au participat la cel putin 10 turnee ATP,
20     // sortati alfabetic dupa nume.
21     System.out.println("a) Jucatori cu cel putin 10 turnee, sortati dupa nume:"
22 );
23     jucatori.stream()
24         .filter(j -> j.getNrTurneeATP() >= 10)
25         .sorted(Comparator.comparing(TennisATP::getNumJucator))
26         .forEach(System.out::println);
27
28     // b) Lista nationalitatilor distincte ale jucatorilor ATP.
29     System.out.println("\nb) Nationalitati distincte:");
30     jucatori.stream()
31         .map(TennisATP::getNationalitateJucator)
32         .distinct()
33         .sorted()
34         .forEach(System.out::println);
35
36     // c) Colectie cu jucatorii care au acumulat intre 1000 si 4000 de puncte.
37     List<TennisATP> intrel1000Si4000 = jucatori.stream()
38         .filter(j -> j.getPuncteATP() >= 1000 && j.getPuncteATP() <= 4000)
39         .collect(Collectors.toList());
40
41     System.out.println("\nc) Jucatori cu puncte intre 1000 si 4000:");
42     intrel1000Si4000.forEach(System.out::println);
43
44     // d) Pentru fiecare nationalitate, numarul de jucatori.
45     System.out.println("\nd) Numar jucatori pentru fiecare nationalitate:");
46     Map<String, Long> frecventeNationalitati = jucatori.stream()
47         .collect(Collectors.groupingBy(
48             TennisATP::getNationalitateJucator,
49             TreeMap::new,
50             Collectors.counting()
51         ));
52     frecventeNationalitati.forEach((nationalitate, nr) ->
53         System.out.println(nationalitate + " -> " + nr));
54 }
55 }

```

2.3. Explicații pentru fiecare cerință

- La punctul (a), filter păstrează doar jucătorii cu minimum 10 turnee, iar sorted îi ordonează după nume.
- La punctul (b), map transformă fiecare jucător în naționalitatea lui, distinct elimină duplicatele, iar sorted sortează alfabetic.
- La punctul (c), collect(Collectors.toList()) creează efectiv o listă nouă cu elementele filtrate.
- La punctul (d), groupingBy grupează jucătorii după naționalitate, iar counting numără câți jucători sunt în fiecare grup.

3. Subiectul III - fișiere text și fire de executare

Cerința: informațiile despre jucători sunt în fișiere text. Fiecare linie are forma:

```
1 numeJucator,nationalitateJucator,pozitieATP,puncteATP,nrTurneeATP
```

Trebuie să citim de la tastatură o naționalitate și să calculăm numărul total de jucători cu acea naționalitate din fișierele:

```
1 jucatori_1.txt
2 jucatori_2.txt
```

Fiecare fișier va fi procesat de un fir separat.

3.1. Clasa firului de execuție

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
5 public class NumaratorNationalitate extends Thread {
6     private final String numeFisier;
7     private final String nationalitateCautata;
8     private int numarJucatori;
9
10    public NumaratorNationalitate(String numeFisier, String nationalitateCautata) {
11        this.numeFisier = numeFisier;
12        this.nationalitateCautata = nationalitateCautata;
13        this.numarJucatori = 0;
14    }
15
16    @Override
17    public void run() {
18        try (BufferedReader br = new BufferedReader(new FileReader(numeFisier))) {
19            String linie;
20
21            while ((linie = br.readLine()) != null) {
22                String[] campuri = linie.split(",");
23
24                if (campuri.length >= 5) {
25                    String nationalitate = campuri[1].trim();
26
27                    if (nationalitate.equalsIgnoreCase(nationalitateCautata.trim()))
28                ) {
29                        numarJucatori++;
30                    }
31                }
32            } catch (IOException e) {
33                System.out.println("Eroare la citirea fisierului " + numeFisier);
34            }
35        }
36
37        public int getNumarJucatori() {
38            return numarJucatori;
39        }
40    }
```

3.2. Programul principal

```
1 import java.util.Scanner;
2
3 public class TestNumaratorNationalitate {
4     public static void main(String[] args) throws InterruptedException {
5         Scanner scanner = new Scanner(System.in);
6
7         System.out.print("Introduceti nationalitatea cautata: ");
8         String nationalitate = scanner.nextLine();
9
10        NumaratorNationalitate fir1 =
11            new NumaratorNationalitate("jucatori_1.txt", nationalitate);
12        NumaratorNationalitate fir2 =
13            new NumaratorNationalitate("jucatori_2.txt", nationalitate);
14
15        fir1.start();
16        fir2.start();
17
18        fir1.join();
19        fir2.join();
20
21        int total = fir1.getNumarJucatori() + fir2.getNumarJucatori();
22
23        System.out.println("Numar total de jucatori cu nationalitatea "
24            + nationalitate + ": " + total);
25    }
26 }
```

3.3. Explicație

- Clasa NumaratorNationalitate extinde Thread, deci fiecare obiect al acestei clase poate rula pe fir separat.
- Metoda run() conține codul care se execută pe firul respectiv.
- Fiecare fir primește un fișier și aceeași naționalitate căutată.
- start() pornește firele în paralel/concurent.
- join() obligă firul principal main să aștepte terminarea firelor fir1 și fir2.
- După join(), rezultatele parțiale sunt sigure și pot fi adunate.

Idee de examen: Dacă apelăm direct run(), nu pornim un fir nou. Pentru execuție pe fir separat trebuie apelat start().

4. Subiectul IV - clasă imutabilă Facultate

Cerința spune că există deja o clasă mutabilă Adresa, cu informații precum stradă, număr, bloc, scară, etaj, apartament, județ și localitate.

Trebuie să definim o clasă imutabilă Facultate care conține:

- denumireFacultate - șir de caractere;
- numarStudenti - număr natural;
- specializari - listă de String;
- adresa - referință spre un obiect de tip Adresa.

4.1. Principiile de imutabilitate

Pentru ca o clasă să fie imutabilă:

1. clasa se declară final, ca să nu poată fi extinsă;
2. toate câmpurile sunt private final;
3. nu există setteri;
4. toate câmpurile se inițializează în constructor;
5. pentru obiectele mutabile primite în constructor se fac copii defensive;
6. getterele nu returnează referințe directe la obiecte mutabile interne.

4.2. Varianta de rezolvare

Presupunem că Adresa are constructor de copiere:

```
1 public Adresa(Adresa altaAdresa) { ... }
```

Atunci clasa Facultate poate fi scrisă astfel:

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.List;
4 import java.util.Objects;
5
6 public final class Facultate {
7     private final String denumireFacultate;
8     private final int numarStudenti;
9     private final List<String> specializari;
10    private final Adresa adresa;
11
12    public Facultate(String denumireFacultate, int numarStudenti,
13                    List<String> specializari, Adresa adresa) {
14        if (denumireFacultate == null) {
15            throw new IllegalArgumentException("Denumirea nu poate fi null.");
16        }
17        if (numarStudenti < 0) {
18            throw new IllegalArgumentException("Numarul de studenti trebuie sa fie
19natural.");
20        }
21        if (specializari == null) {
22            throw new IllegalArgumentException("Lista de specializari nu poate fi
23null.");
24        }
25        if (adresa == null) {
26            throw new IllegalArgumentException("Adresa nu poate fi null.");
27        }
28    }
29}
```

```

25     }
26
27     this.denumireFacultate = denumireFacultate;
28     this.numarStudenti = numarStudenti;
29
30     // String este imutabil, dar lista este mutabila.
31     // De aceea copiem lista si o facem nemodificabila.
32     this.specializari = Collections.unmodifiableList(new ArrayList<>(
specializari));
33
34     // Adresa este mutabila, deci facem copie defensiva.
35     this.adresa = new Adresa(adresa);
36 }
37
38 public String getDenumireFacultate() {
39     return denumireFacultate;
40 }
41
42 public int getNumarStudenti() {
43     return numarStudenti;
44 }
45
46 public List<String> getSpecializari() {
47     // Este sigur sa returnam lista, deoarece este nemodificabila,
48     // iar String este imutabil.
49     return specializari;
50 }
51
52 public Adresa getAdresa() {
53     // Nu returnam referinta interna, ci o copie.
54     return new Adresa(adresa);
55 }
56
57 @Override
58 public String toString() {
59     return "Facultate{" +
60         "denumireFacultate='" + denumireFacultate + '\'' +
61         ", numarStudenti=" + numarStudenti +
62         ", specializari=" + specializari +
63         ", adresa=" + adresa +
64         '}';
65 }
66
67 @Override
68 public boolean equals(Object obj) {
69     if (this == obj) {
70         return true;
71     }
72     if (!(obj instanceof Facultate)) {
73         return false;
74     }
75     Facultate other = (Facultate) obj;
76     return numarStudenti == other.numarStudenti
77         && Objects.equals(denumireFacultate, other.denumireFacultate)
78         && Objects.equals(specializari, other.specializari)
79         && Objects.equals(adresa, other.adresa);
80 }
81
82 @Override
83 public int hashCode() {
84     return Objects.hash(denumireFacultate, numarStudenti, specializari, adresa)
;

```

```
85     }  
86 }
```

4.3. De ce sunt necesare copiile defensive?

Dacă am scrie:

```
1 this.adresa = adresa;
```

atunci clasa nu ar mai fi imutabilă. Cineva din exterior ar putea modifica obiectul `Adresa` primit în constructor, iar modificarea s-ar vedea în interiorul obiectului `Facultate`.

La fel, dacă am scrie:

```
1 this.specializari = specializari;
```

lista internă ar putea fi modificată din exterior prin aceeași referință.

Corect este:

```
1 this.specializari = Collections.unmodifiableList(new ArrayList<>(specializari));  
2 this.adresa = new Adresa(adresa);
```

4.4. Dacă `Adresa` nu are constructor de copiere

Dacă nu există `new Adresa(adresa)`, atunci se construiește explicit o adresă nouă folosind getterele clasei `Adresa`:

```
1 this.adresa = new Adresa(  
2     adresa.getStrada(),  
3     adresa.getNumar(),  
4     adresa.getBloc(),  
5     adresa.getScara(),  
6     adresa.getEtaj(),  
7     adresa.getApartament(),  
8     adresa.getJudet(),  
9     adresa.getLocalitate()  
10 );
```

Și în getter se procedează la fel.

Concluzie pentru examen

Cele mai importante idei din acest subiect sunt:

- Metodele `final` nu pot fi suprascrise.
- La overriding nu putem reduce nivelul de acces.
- O metodă de instanță nu poate fi redefinită ca metodă statică.
- `HashSet` folosește împreună `hashCode` și `equals`.
- La upcasting, câmpurile se aleg după tipul referinței, iar metodele suprascrise după tipul real al obiectului.
- `finally` se execută chiar și când se aruncă o excepție în `catch`.
- O interfață funcțională are o singură metodă abstractă.
- Pentru imutabilitate, obiectele mutabile trebuie copiate defensiv în constructor și în getter.